

- Chirag Miglani
- 1/23/SET/BCS/414
- 6AIMLB1
- DATE: 05/03/2026
- EXPERIMENT: 07

House Price Prediction

Training/Testing 80/20

```

# Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam

# Load Dataset
data = pd.read_csv("/content/data.csv")

print(data.head())
print(data.columns)

# Drop unnecessary columns
data = data.drop(['date', 'street', 'city', 'statezip', 'country'], axis=1)

# Define X and Y
X = data.drop('price', axis=1)
y = data['price']

# Train Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Normalization
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# -----
# Main Model Training
# -----

model = Sequential()

model.add(Dense(128, activation='relu', input_shape=(X_train.shape[1],)))
model.add(Dense(64, activation='relu'))
model.add(Dense(32, activation='relu'))
model.add(Dense(1))

optimizer = Adam(learning_rate=0.001)

model.compile(
    optimizer=optimizer,
    loss='mean_squared_error',
    metrics=['mae']
)

# Model Summary
model.summary()

# Train Model
history = model.fit(
    X_train,
    y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2
)

```

```

        validation_split=0.2
    )

    # Evaluate Model
    loss, mae = model.evaluate(X_test, y_test)

    print("Test Loss:", loss)
    print("Test MAE:", mae)

    # -----
    # Visualization 1: Loss vs Epoch
    # -----

    plt.figure()
    plt.plot(history.history['loss'], label="Training Loss")
    plt.plot(history.history['val_loss'], label="Validation Loss")
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.title("Loss vs Epoch")
    plt.legend()
    plt.show()

    # -----
    # Visualization 2: MAE vs Epoch
    # -----

    plt.figure()
    plt.plot(history.history['mae'], label="Training MAE")
    plt.plot(history.history['val_mae'], label="Validation MAE")
    plt.xlabel("Epoch")
    plt.ylabel("MAE")
    plt.title("MAE vs Epoch")
    plt.legend()
    plt.show()

    # -----
    # Learning Rate Experiment
    # -----

    learning_rates = [0.0001, 0.001, 0.01, 0.1]

    plt.figure()

    for lr in learning_rates:

        temp_model = Sequential()
        temp_model.add(Dense(128, activation='relu', input_shape=(X_train.shape[1],)))
        temp_model.add(Dense(64, activation='relu'))
        temp_model.add(Dense(32, activation='relu'))
        temp_model.add(Dense(1))

        optimizer = Adam(learning_rate=lr)

        temp_model.compile(
            optimizer=optimizer,
            loss='mean_squared_error',
            metrics=['mae']
        )

        history_lr = temp_model.fit(
            X_train,
            y_train,
            epochs=20,
            batch_size=32,
            verbose=0
        )

        plt.plot(history_lr.history['loss'], label=f"LR = {lr}")

    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.title("Loss vs Epoch for Different Learning Rates")
    plt.legend()
    plt.show()

    # -----
    # MAE vs Epoch for Learning Rates
    # -----

```

```
plt.figure()

for lr in learning_rates:

    temp_model = Sequential()
    temp_model.add(Dense(128, activation='relu', input_shape=(X_train.shape[1],)))
    temp_model.add(Dense(64, activation='relu'))
    temp_model.add(Dense(32, activation='relu'))
    temp_model.add(Dense(1))

    optimizer = Adam(learning_rate=lr)

    temp_model.compile(
        optimizer=optimizer,
        loss='mean_squared_error',
        metrics=['mae']
    )

    history_lr = temp_model.fit(
        X_train,
        y_train,
        epochs=20,
        batch_size=32,
        verbose=0
    )

    plt.plot(history_lr.history['mae'], label=f"LR = {lr}")

plt.xlabel("Epoch")
plt.ylabel("MAE")
plt.title("MAE vs Epoch for Different Learning Rates")
plt.legend()
plt.show()
```



```

    date      price  bedrooms  bathrooms  sqft_living  sqft_lot  \
0  2014-05-02 00:00:00  313000.0      3.0      1.50      1340      7912
1  2014-05-02 00:00:00  2384000.0      5.0      2.50      3650      9050
2  2014-05-02 00:00:00  342000.0      3.0      2.00      1930      11947
3  2014-05-02 00:00:00  420000.0      3.0      2.25      2000      8030
4  2014-05-02 00:00:00  550000.0      4.0      2.50      1940      10500

    floors  waterfront  view  condition  sqft_above  sqft_basement  yr_built  \
0      1.5          0      0          3      1340          0      1955
1      2.0          0      4          5      3370          280      1921
2      1.0          0      0          4      1930          0      1966
3      1.0          0      0          4      1000        1000      1963
4      1.0          0      0          4      1140          800      1976

    yr_renovated      street      city  statezip  country
0          2005      18810 Densmore Ave N  Shoreline  WA  98133      USA
1              0          709 W Blaine St    Seattle  WA  98119      USA
2              0  26206-26214 143rd Ave SE      Kent  WA  98042      USA
3              0          857 170th Pl NE  Bellevue  WA  98008      USA
4          1992          9105 170th Ave NE  Redmond  WA  98052      USA
Index(['date', 'price', 'bedrooms', 'bathrooms', 'sqft_living', 'sqft_lot',
      'floors', 'waterfront', 'view', 'condition', 'sqft_above',
      'sqft_basement', 'yr_built', 'yr_renovated', 'street', 'city',
      'statezip', 'country'],
      dtype='object')
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_27"
```

Layer (type)	Output Shape	Param #
dense_108 (Dense)	(None, 128)	1,664
dense_109 (Dense)	(None, 64)	8,256
dense_110 (Dense)	(None, 32)	2,080
dense_111 (Dense)	(None, 1)	33

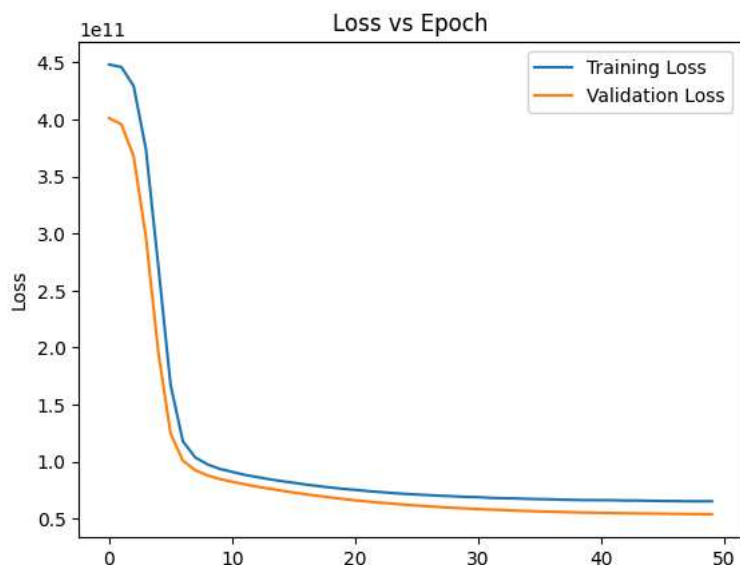
```

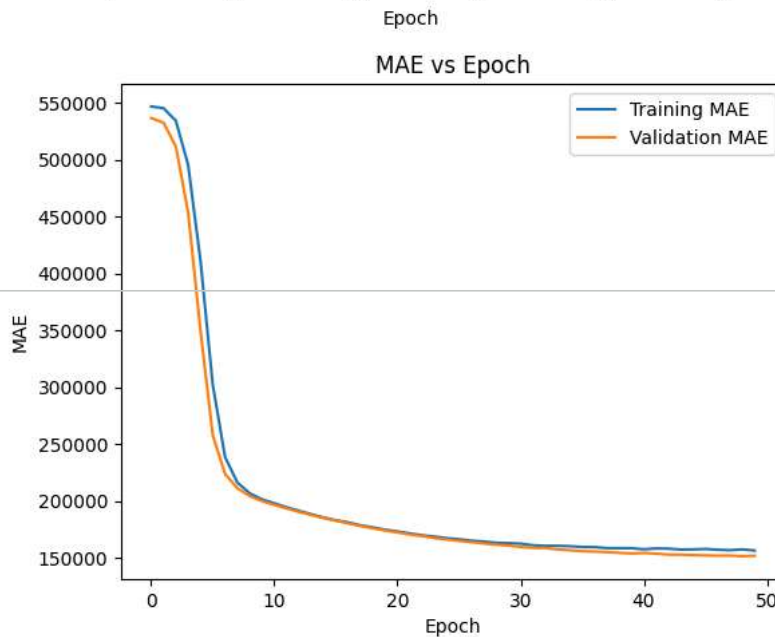
Total params: 12,033 (47.00 KB)
Trainable params: 12,033 (47.00 KB)
Non-trainable params: 0 (0.00 B)
Epoch 1/50
92/92 ━━━━━━━━━━━ 2s 8ms/step - loss: 444417540096.0000 - mae: 545634.5000 - val_loss: 401250516992.0000 - val_mae: 5
Epoch 2/50
92/92 ━━━━━━━━━━━ 1s 5ms/step - loss: 404833959936.0000 - mae: 539002.3750 - val_loss: 395832000512.0000 - val_mae: 5
Epoch 3/50
92/92 ━━━━━━━━━━━ 1s 6ms/step - loss: 436243038208.0000 - mae: 542109.5625 - val_loss: 367721742336.0000 - val_mae: 5
Epoch 4/50
92/92 ━━━━━━━━━━━ 0s 5ms/step - loss: 381512777728.0000 - mae: 501930.2500 - val_loss: 296714600448.0000 - val_mae: 4
Epoch 5/50
92/92 ━━━━━━━━━━━ 0s 3ms/step - loss: 325442732032.0000 - mae: 444537.1875 - val_loss: 196273274880.0000 - val_mae: 3
Epoch 6/50
92/92 ━━━━━━━━━━━ 0s 3ms/step - loss: 181547663360.0000 - mae: 329734.8438 - val_loss: 124889923584.0000 - val_mae: 2
Epoch 7/50
92/92 ━━━━━━━━━━━ 0s 4ms/step - loss: 118342172672.0000 - mae: 239802.3281 - val_loss: 100953538560.0000 - val_mae: 2
Epoch 8/50
92/92 ━━━━━━━━━━━ 0s 3ms/step - loss: 120586559488.0000 - mae: 219018.9375 - val_loss: 92540567552.0000 - val_mae: 21
Epoch 9/50
92/92 ━━━━━━━━━━━ 0s 3ms/step - loss: 95544410112.0000 - mae: 209150.1719 - val_loss: 88017887232.0000 - val_mae: 204
Epoch 10/50
92/92 ━━━━━━━━━━━ 0s 3ms/step - loss: 92802473984.0000 - mae: 200160.3125 - val_loss: 84844503040.0000 - val_mae: 199
Epoch 11/50
92/92 ━━━━━━━━━━━ 0s 3ms/step - loss: 113457528832.0000 - mae: 205099.2031 - val_loss: 82430943232.0000 - val_mae: 19
Epoch 12/50
92/92 ━━━━━━━━━━━ 0s 3ms/step - loss: 86723190784.0000 - mae: 196575.2500 - val_loss: 80310853632.0000 - val_mae: 193
Epoch 13/50
92/92 ━━━━━━━━━━━ 0s 3ms/step - loss: 75890745344.0000 - mae: 185287.6094 - val_loss: 78176272384.0000 - val_mae: 190
Epoch 14/50
92/92 ━━━━━━━━━━━ 0s 3ms/step - loss: 90238173184.0000 - mae: 187504.6875 - val_loss: 76503220224.0000 - val_mae: 187
Epoch 15/50
92/92 ━━━━━━━━━━━ 0s 3ms/step - loss: 84624506880.0000 - mae: 187187.0312 - val_loss: 74667884544.0000 - val_mae: 185
Epoch 16/50
92/92 ━━━━━━━━━━━ 0s 3ms/step - loss: 77559767040.0000 - mae: 184100.2344 - val_loss: 72976949248.0000 - val_mae: 182
Epoch 17/50
92/92 ━━━━━━━━━━━ 0s 3ms/step - loss: 85630066688.0000 - mae: 185342.9531 - val_loss: 71391051776.0000 - val_mae: 180
Epoch 18/50
92/92 ━━━━━━━━━━━ 0s 3ms/step - loss: 76424970240.0000 - mae: 180586.9062 - val_loss: 70084050944.0000 - val_mae: 178
Epoch 19/50
92/92 ━━━━━━━━━━━ 0s 3ms/step - loss: 77260890112.0000 - mae: 174090.4688 - val_loss: 68627988480.0000 - val_mae: 176
Epoch 20/50
92/92 ━━━━━━━━━━━ 0s 3ms/step - loss: 71121108992.0000 - mae: 166009.3906 - val_loss: 67429478400.0000 - val_mae: 174
Epoch 21/50
92/92 ━━━━━━━━━━━ 0s 3ms/step - loss: 75815919616.0000 - mae: 173951.9844 - val_loss: 66167840768.0000 - val_mae: 172
Epoch 22/50
```

```

Epoch 22/50
92/92 1s 5ms/step - loss: 88315461632.0000 - mae: 175490.8594 - val_loss: 65194483712.0000 - val_mae: 1704
Epoch 23/50
92/92 0s 4ms/step - loss: 70657449984.0000 - mae: 167333.1250 - val_loss: 64126115840.0000 - val_mae: 169
Epoch 24/50
92/92 0s 3ms/step - loss: 85856018432.0000 - mae: 174435.0938 - val_loss: 63325065216.0000 - val_mae: 1674
Epoch 25/50
92/92 0s 3ms/step - loss: 81684692992.0000 - mae: 171708.6562 - val_loss: 62397214720.0000 - val_mae: 1664
Epoch 26/50
92/92 0s 3ms/step - loss: 79051055104.0000 - mae: 170221.8594 - val_loss: 61607600128.0000 - val_mae: 1654
Epoch 27/50
92/92 0s 3ms/step - loss: 71361724416.0000 - mae: 166072.0469 - val_loss: 60889079808.0000 - val_mae: 1634
Epoch 28/50
92/92 0s 3ms/step - loss: 67094020096.0000 - mae: 162357.2188 - val_loss: 60183732224.0000 - val_mae: 1624
Epoch 29/50
92/92 1s 3ms/step - loss: 79137996800.0000 - mae: 168740.2500 - val_loss: 59598753792.0000 - val_mae: 1614
Epoch 30/50
92/92 0s 3ms/step - loss: 73257893888.0000 - mae: 162618.5156 - val_loss: 59035676672.0000 - val_mae: 1604
Epoch 31/50
92/92 0s 3ms/step - loss: 77777690624.0000 - mae: 168325.2031 - val_loss: 58482515968.0000 - val_mae: 1594
Epoch 32/50
92/92 0s 3ms/step - loss: 63598821376.0000 - mae: 160689.8438 - val_loss: 58041720832.0000 - val_mae: 1584
Epoch 33/50
92/92 0s 5ms/step - loss: 68599242752.0000 - mae: 162507.6250 - val_loss: 57629114368.0000 - val_mae: 1584
Epoch 34/50
92/92 1s 5ms/step - loss: 69756084224.0000 - mae: 162175.3750 - val_loss: 57191276544.0000 - val_mae: 1574
Epoch 35/50
92/92 1s 6ms/step - loss: 66304798720.0000 - mae: 156866.7344 - val_loss: 56805105664.0000 - val_mae: 1564
Epoch 36/50
92/92 0s 5ms/step - loss: 73551233024.0000 - mae: 163275.6719 - val_loss: 56452108288.0000 - val_mae: 1554
Epoch 37/50
92/92 1s 6ms/step - loss: 64665677824.0000 - mae: 161620.9688 - val_loss: 56166473728.0000 - val_mae: 1554
Epoch 38/50
92/92 0s 5ms/step - loss: 74577813504.0000 - mae: 165968.9375 - val_loss: 55878701056.0000 - val_mae: 1554
Epoch 39/50
92/92 0s 3ms/step - loss: 64498999296.0000 - mae: 158949.4844 - val_loss: 55630340096.0000 - val_mae: 1544
Epoch 40/50
92/92 0s 3ms/step - loss: 63177097216.0000 - mae: 160486.7500 - val_loss: 55401021440.0000 - val_mae: 1534
Epoch 41/50
92/92 0s 3ms/step - loss: 70164463616.0000 - mae: 160608.5938 - val_loss: 55217401856.0000 - val_mae: 1544
Epoch 42/50
92/92 0s 3ms/step - loss: 69598658560.0000 - mae: 161726.5938 - val_loss: 55006670848.0000 - val_mae: 1534
Epoch 43/50
92/92 0s 3ms/step - loss: 66085302272.0000 - mae: 156198.7031 - val_loss: 54809534464.0000 - val_mae: 1524
Epoch 44/50
92/92 0s 3ms/step - loss: 60575551488.0000 - mae: 157126.5625 - val_loss: 54696812544.0000 - val_mae: 1524
Epoch 45/50
92/92 0s 3ms/step - loss: 66813808640.0000 - mae: 161006.1719 - val_loss: 54543437824.0000 - val_mae: 1524
Epoch 46/50
92/92 0s 3ms/step - loss: 58888667136.0000 - mae: 151453.4531 - val_loss: 54452510720.0000 - val_mae: 1524
Epoch 47/50
92/92 0s 3ms/step - loss: 74701864960.0000 - mae: 160819.8281 - val_loss: 54298566656.0000 - val_mae: 1524
Epoch 48/50
92/92 0s 3ms/step - loss: 59653263360.0000 - mae: 153790.9219 - val_loss: 54169174016.0000 - val_mae: 1524
Epoch 49/50
92/92 0s 3ms/step - loss: 66400022528.0000 - mae: 162054.4219 - val_loss: 54096646144.0000 - val_mae: 1514
Epoch 50/50
92/92 0s 3ms/step - loss: 67288346624.0000 - mae: 157071.6094 - val_loss: 54008913920.0000 - val_mae: 1514
29/29 0s 3ms/step - loss: 187561295872.0000 - mae: 175775.1875
Test Loss: 985583910912.0
Test MAE: 206377.21875

```

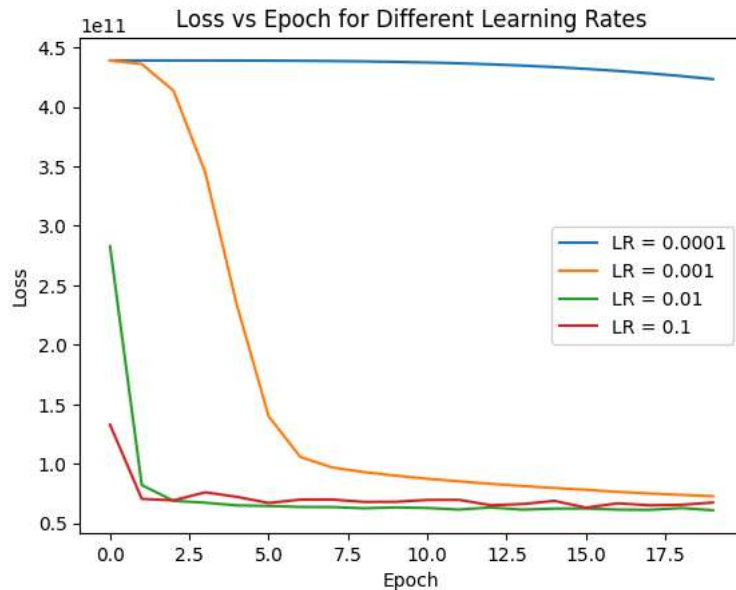




```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape` to `input_dir`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape` to `input_dir`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape` to `input_dir`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape` to `input_dir`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

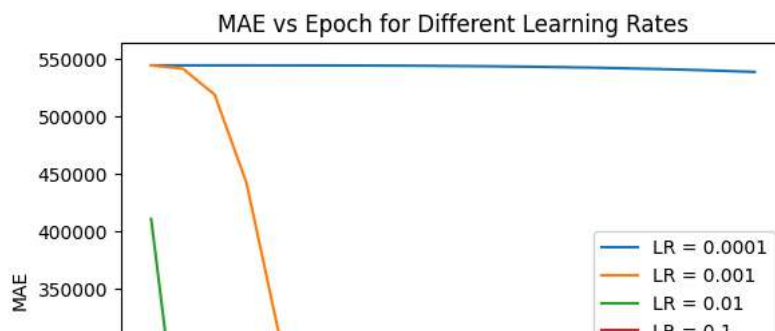
```



```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape` to `input_dir`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape` to `input_dir`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape` to `input_dir`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape` to `input_dir`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```



300000
Training/Testing 70/30
250000

```
# Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam

# Load Dataset
data = pd.read_csv("/content/data.csv")

print(data.head())
print(data.columns)

# Drop unnecessary columns
data = data.drop(['date', 'street', 'city', 'statezip', 'country'], axis=1)

# Define X and Y
X = data.drop('price', axis=1)
y = data['price']

# Train Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Normalization
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# -----
# Main Model Training
# -----

model = Sequential()

model.add(Dense(128, activation='relu', input_shape=(X_train.shape[1],)))
model.add(Dense(64, activation='relu'))
model.add(Dense(32, activation='relu'))
model.add(Dense(1))

optimizer = Adam(learning_rate=0.001)

model.compile(
    optimizer=optimizer,
    loss='mean_squared_error',
    metrics=['mae']
)

# Model Summary
model.summary()

# Train Model
history = model.fit(
    X_train,
    y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2
)

# Evaluate Model
loss, mae = model.evaluate(X_test, y_test)

print("Test Loss:", loss)
print("Test MAE:", mae)

# -----
# Visualization 1: Loss vs Epoch
# -----
```

```

plt.figure()
plt.plot(history.history['loss'], label="Training Loss")
plt.plot(history.history['val_loss'], label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Loss vs Epoch")
plt.legend()
plt.show()

# -----
# Visualization 2: MAE vs Epoch
# -----

plt.figure()
plt.plot(history.history['mae'], label="Training MAE")
plt.plot(history.history['val_mae'], label="Validation MAE")
plt.xlabel("Epoch")
plt.ylabel("MAE")
plt.title("MAE vs Epoch")
plt.legend()
plt.show()

# -----
# Learning Rate Experiment
# -----

learning_rates = [0.0001, 0.001, 0.01, 0.1]

plt.figure()

for lr in learning_rates:

    temp_model = Sequential()
    temp_model.add(Dense(128, activation='relu', input_shape=(X_train.shape[1],)))
    temp_model.add(Dense(64, activation='relu'))
    temp_model.add(Dense(32, activation='relu'))
    temp_model.add(Dense(1))

    optimizer = Adam(learning_rate=lr)

    temp_model.compile(
        optimizer=optimizer,
        loss='mean_squared_error',
        metrics=['mae']
    )

    history_lr = temp_model.fit(
        X_train,
        y_train,
        epochs=20,
        batch_size=32,
        verbose=0
    )

    plt.plot(history_lr.history['loss'], label=f"LR = {lr}")

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Loss vs Epoch for Different Learning Rates")
plt.legend()
plt.show()

# -----
# MAE vs Epoch for Learning Rates
# -----

plt.figure()

for lr in learning_rates:

    temp_model = Sequential()
    temp_model.add(Dense(128, activation='relu', input_shape=(X_train.shape[1],)))
    temp_model.add(Dense(64, activation='relu'))
    temp_model.add(Dense(32, activation='relu'))
    temp_model.add(Dense(1))

    optimizer = Adam(learning_rate=lr)

```

```
temp_model.compile(
    optimizer=optimizer,
    loss='mean_squared_error',
    metrics=['mae']
)

history_lr = temp_model.fit(
    X_train,
    y_train,
    epochs=20,
    batch_size=32,
    verbose=0
)

plt.plot(history_lr.history['mae'], label=f"LR = {lr}")

plt.xlabel("Epoch")
plt.ylabel("MAE")
plt.title("MAE vs Epoch for Different Learning Rates")
plt.legend()
plt.show()
```



```

    date      price  bedrooms  bathrooms  sqft_living  sqft_lot  \
0  2014-05-02 00:00:00  313000.0      3.0      1.50      1340      7912
1  2014-05-02 00:00:00  2384000.0      5.0      2.50      3650      9050
2  2014-05-02 00:00:00  342000.0      3.0      2.00      1930      11947
3  2014-05-02 00:00:00  420000.0      3.0      2.25      2000      8030
4  2014-05-02 00:00:00  550000.0      4.0      2.50      1940      10500

    floors  waterfront  view  condition  sqft_above  sqft_basement  yr_built  \
0      1.5          0      0          3      1340          0      1955
1      2.0          0      4          5      3370          280      1921
2      1.0          0      0          4      1930          0      1966
3      1.0          0      0          4      1000        1000      1963
4      1.0          0      0          4      1140          800      1976

    yr_renovated      street      city  statezip  country
0          2005      18810 Densmore Ave N  Shoreline  WA  98133      USA
1              0          709 W Blaine St  Seattle  WA  98119      USA
2              0  26206-26214 143rd Ave SE      Kent  WA  98042      USA
3              0          857 170th Pl NE  Bellevue  WA  98008      USA
4          1992          9105 170th Ave NE  Redmond  WA  98052      USA
Index(['date', 'price', 'bedrooms', 'bathrooms', 'sqft_living', 'sqft_lot',
      'floors', 'waterfront', 'view', 'condition', 'sqft_above',
      'sqft_basement', 'yr_built', 'yr_renovated', 'street', 'city',
      'statezip', 'country'],
      dtype='object')
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_36"
```

Layer (type)	Output Shape	Param #
dense_144 (Dense)	(None, 128)	1,664
dense_145 (Dense)	(None, 64)	8,256
dense_146 (Dense)	(None, 32)	2,080
dense_147 (Dense)	(None, 1)	33

```

Total params: 12,033 (47.00 KB)
Trainable params: 12,033 (47.00 KB)
Non-trainable params: 0 (0.00 B)
Epoch 1/50
81/81 ━━━━━━━━━━━ 2s 6ms/step - loss: 472142184448.0000 - mae: 554056.6250 - val_loss: 406683058176.0000 - val_mae: 5
Epoch 2/50
81/81 ━━━━━━━━━━━ 0s 3ms/step - loss: 406354952192.0000 - mae: 527402.5000 - val_loss: 404312162304.0000 - val_mae: 5
Epoch 3/50
81/81 ━━━━━━━━━━━ 0s 3ms/step - loss: 429584646144.0000 - mae: 546322.0000 - val_loss: 390968868864.0000 - val_mae: 5
Epoch 4/50
81/81 ━━━━━━━━━━━ 0s 6ms/step - loss: 439985831936.0000 - mae: 529163.1875 - val_loss: 351895683072.0000 - val_mae: 4
Epoch 5/50
81/81 ━━━━━━━━━━━ 1s 6ms/step - loss: 354946187264.0000 - mae: 481295.0625 - val_loss: 274669207552.0000 - val_mae: 4
Epoch 6/50
81/81 ━━━━━━━━━━━ 1s 6ms/step - loss: 284740714496.0000 - mae: 420277.6875 - val_loss: 182897180672.0000 - val_mae: 3
Epoch 7/50
81/81 ━━━━━━━━━━━ 0s 5ms/step - loss: 210917933056.0000 - mae: 318796.5625 - val_loss: 120736645120.0000 - val_mae: 2
Epoch 8/50
81/81 ━━━━━━━━━━━ 1s 6ms/step - loss: 125212590080.0000 - mae: 243757.8594 - val_loss: 97184284672.0000 - val_mae: 21
Epoch 9/50
81/81 ━━━━━━━━━━━ 1s 5ms/step - loss: 107952259072.0000 - mae: 219646.0000 - val_loss: 89981943808.0000 - val_mae: 20
Epoch 10/50
81/81 ━━━━━━━━━━━ 0s 3ms/step - loss: 94663376896.0000 - mae: 202163.6094 - val_loss: 86416785408.0000 - val_mae: 20
Epoch 11/50
81/81 ━━━━━━━━━━━ 0s 3ms/step - loss: 87905132544.0000 - mae: 204325.5625 - val_loss: 83645276160.0000 - val_mae: 198
Epoch 12/50
81/81 ━━━━━━━━━━━ 0s 4ms/step - loss: 103163764736.0000 - mae: 207660.8594 - val_loss: 81177198592.0000 - val_mae: 19
Epoch 13/50
81/81 ━━━━━━━━━━━ 0s 4ms/step - loss: 80653688832.0000 - mae: 189623.6719 - val_loss: 79218532352.0000 - val_mae: 192
Epoch 14/50
81/81 ━━━━━━━━━━━ 0s 3ms/step - loss: 96662781952.0000 - mae: 199328.3594 - val_loss: 77085696000.0000 - val_mae: 189
Epoch 15/50
81/81 ━━━━━━━━━━━ 0s 3ms/step - loss: 89164267520.0000 - mae: 188721.2812 - val_loss: 75354660864.0000 - val_mae: 186
Epoch 16/50
81/81 ━━━━━━━━━━━ 0s 4ms/step - loss: 83598352384.0000 - mae: 187953.2188 - val_loss: 73748971520.0000 - val_mae: 184
Epoch 17/50
81/81 ━━━━━━━━━━━ 1s 7ms/step - loss: 83962806272.0000 - mae: 182660.6094 - val_loss: 72099725312.0000 - val_mae: 181
Epoch 18/50
81/81 ━━━━━━━━━━━ 0s 5ms/step - loss: 81189371904.0000 - mae: 187667.7344 - val_loss: 70636003328.0000 - val_mae: 179
Epoch 19/50
81/81 ━━━━━━━━━━━ 1s 7ms/step - loss: 85682831360.0000 - mae: 181564.9219 - val_loss: 69210882048.0000 - val_mae: 177
Epoch 20/50
81/81 ━━━━━━━━━━━ 1s 7ms/step - loss: 88077090816.0000 - mae: 177811.9219 - val_loss: 67924967424.0000 - val_mae: 175
Epoch 21/50
81/81 ━━━━━━━━━━━ 1s 6ms/step - loss: 78919303168.0000 - mae: 175008.6406 - val_loss: 66749415424.0000 - val_mae: 173
Epoch 22/50
```

```

Epoch 22/50
81/81 — 1s 6ms/step - loss: 79025225728.0000 - mae: 174603.6562 - val_loss: 65500823552.0000 - val_mae: 1724
Epoch 23/50
81/81 — 1s 7ms/step - loss: 81709113344.0000 - mae: 175370.5312 - val_loss: 64416342016.0000 - val_mae: 1704
Epoch 24/50
81/81 — 1s 6ms/step - loss: 91471544320.0000 - mae: 179532.8125 - val_loss: 63430504448.0000 - val_mae: 1684
Epoch 25/50
81/81 — 1s 7ms/step - loss: 75085922304.0000 - mae: 172907.3594 - val_loss: 62454697984.0000 - val_mae: 1674
Epoch 26/50
81/81 — 0s 3ms/step - loss: 82988892160.0000 - mae: 180298.4531 - val_loss: 61574918144.0000 - val_mae: 1664
Epoch 27/50
81/81 — 0s 3ms/step - loss: 81055408128.0000 - mae: 169612.5938 - val_loss: 60784316416.0000 - val_mae: 1644
Epoch 28/50
81/81 — 0s 4ms/step - loss: 77728694272.0000 - mae: 170523.0312 - val_loss: 60014530560.0000 - val_mae: 1634
Epoch 29/50
81/81 — 0s 3ms/step - loss: 67096596480.0000 - mae: 161567.7812 - val_loss: 59261808640.0000 - val_mae: 1624
Epoch 30/50
81/81 — 0s 3ms/step - loss: 67181813760.0000 - mae: 161546.0000 - val_loss: 58557239296.0000 - val_mae: 1614
Epoch 31/50
81/81 — 0s 4ms/step - loss: 76904841216.0000 - mae: 167320.9375 - val_loss: 57927864320.0000 - val_mae: 1604
Epoch 32/50
81/81 — 0s 3ms/step - loss: 72635023360.0000 - mae: 163353.2656 - val_loss: 57396367360.0000 - val_mae: 1594
Epoch 33/50
81/81 — 0s 6ms/step - loss: 63579545600.0000 - mae: 155601.0312 - val_loss: 56868868096.0000 - val_mae: 1584
Epoch 34/50
81/81 — 1s 6ms/step - loss: 66826379264.0000 - mae: 162331.8281 - val_loss: 56378777600.0000 - val_mae: 1584
Epoch 35/50
81/81 — 0s 5ms/step - loss: 60128915456.0000 - mae: 157663.9062 - val_loss: 55931338752.0000 - val_mae: 1574
Epoch 36/50
81/81 — 1s 7ms/step - loss: 69689131008.0000 - mae: 159065.3594 - val_loss: 55457075200.0000 - val_mae: 1574
Epoch 37/50
81/81 — 1s 6ms/step - loss: 101782495232.0000 - mae: 173017.7344 - val_loss: 55148924928.0000 - val_mae: 1564
Epoch 38/50
81/81 — 1s 8ms/step - loss: 63424188416.0000 - mae: 159971.1250 - val_loss: 54859120640.0000 - val_mae: 1554
Epoch 39/50
81/81 — 1s 6ms/step - loss: 80097427456.0000 - mae: 167541.0312 - val_loss: 54453620736.0000 - val_mae: 1554
Epoch 40/50
81/81 — 1s 6ms/step - loss: 63744040960.0000 - mae: 159280.4062 - val_loss: 54155075584.0000 - val_mae: 1554
Epoch 41/50
81/81 — 1s 7ms/step - loss: 73650503680.0000 - mae: 161945.0156 - val_loss: 53858570240.0000 - val_mae: 1544
Epoch 42/50
81/81 — 1s 7ms/step - loss: 76809175040.0000 - mae: 165836.7812 - val_loss: 53649014784.0000 - val_mae: 1534
Epoch 43/50
81/81 — 1s 7ms/step - loss: 64205254656.0000 - mae: 156270.2500 - val_loss: 53406334976.0000 - val_mae: 1534
Epoch 44/50
81/81 — 1s 6ms/step - loss: 63779672064.0000 - mae: 159256.7656 - val_loss: 53181734912.0000 - val_mae: 1534
Epoch 45/50
81/81 — 0s 3ms/step - loss: 65344663552.0000 - mae: 160204.2812 - val_loss: 52969902080.0000 - val_mae: 1534
Epoch 46/50
81/81 — 0s 4ms/step - loss: 61102419968.0000 - mae: 153838.0469 - val_loss: 52859641856.0000 - val_mae: 1524
Epoch 47/50
81/81 — 0s 5ms/step - loss: 57735647232.0000 - mae: 155288.2344 - val_loss: 52704108544.0000 - val_mae: 1524
Epoch 48/50
81/81 — 0s 3ms/step - loss: 70836445184.0000 - mae: 158810.9688 - val_loss: 52518985728.0000 - val_mae: 1524
Epoch 49/50
81/81 — 0s 4ms/step - loss: 66106941440.0000 - mae: 157373.4688 - val_loss: 52346732544.0000 - val_mae: 1524
Epoch 50/50
81/81 — 0s 4ms/step - loss: 67717947392.0000 - mae: 160655.8125 - val_loss: 52237209600.0000 - val_mae: 1524
44/44 — 0s 2ms/step - loss: 382919835648.0000 - mae: 183645.7344
Test Loss: 677498650624.0
Test MAE: 191569.1875

```

